

03 - Extraccion a Helpers

Ejercicio 03 — Extracción de código repetido a Helpers ★ ★

Foco: modularización, separación de responsabilidades **Tiempo estimado:** 1 clase [← Volver al README](#)

Objetivo

Que uses la IA para **extraer lógica repetida a funciones reutilizables (helpers)**. A diferencia del ejercicio 02 (que ataca la duplicación entre entidades), este ataca la duplicación **dentro de una misma capa**: el patrón que se repite endpoint por endpoint.

💡 Dato del proyecto: ya existe `src/helpers/log-helper.js` y dos repositorios que delegan el manejo del `Pool` a una clase `DbPg` (`db-pg.js`). O sea: **la idea de "extraer a un helper" ya está plantada en el código**. Tu trabajo es llevarla más lejos.

El problema

Abrí `alumnos-controller.js` y mirá cuántas veces se repite **esto**:

```
try {
  const algo = await currentService.loQueSea();
  if (algo !== null) {
    res.status(StatusCode.OK).json(algo);
  } else {
    res.status(StatusCode.NOT_FOUND).send(`No se encontro...`);
  }
} catch (error) {
  console.log(error);
  res.status(StatusCode.INTERNAL_SERVER_ERROR).send(`Error: ${error.message}`);
}
```

Está en **cada uno de los 5 endpoints**, y otra vez en `cursos-controller.js` (y en `materias-controller.js`, si lo creaste en el ejercicio 01). Candidatos a helper:

Estos dos son los que tenés que hacer (cada uno ejercita un tipo distinto de extracción):

-  **Helper de respuestas** (`responderOk` , `responderNotFound` , `responderError` ...) para unificar el formato de las respuestas HTTP que hoy está copiado en **cada endpoint** de los controllers.
-  **Helper de fechas**: sacar `calcularEdad` / `agregarEdad` , que hoy viven **adentro de** `alumnos-service.js` , a un helper reutilizable. Ese cálculo no es lógica de "alumnos": es una utilidad de fechas que mañana podrían usar otras entidades. Tu trabajo es **darte cuenta de que no pertenece al service** y mudarlo a `src/helpers/` .

La idea: extraer lo repetido a un método

Volvé a mirar el bloque de arriba. Fijate que pedazos como este aparecen **igual** en endpoint tras endpoint, en alumnos, en cursos, en materias:

```
res.status(StatusCode.NOT_FOUND).send(`No se encontro...`);
```

Extraer a un helper es exactamente eso: agarrar ese pedacito que se repite y ponerlo en **una función con nombre** (algo del estilo `responderNotFound(res, mensaje)`) que viva en `src/helpers/` . Después, en cada endpoint, en vez de escribir el `res.status(...).send(...)` a mano, **llamás a esa función**.

Si lo hacés bien, deberías lograr que:

- el endpoint quede **más corto** y más fácil de leer, y
- el "qué status code corresponde a cada caso" quede en **un solo lugar** (el helper), no copiado por todos lados.

 **Cómo lo implementás exactamente —qué funciones creás, cómo se llaman, qué reciben y devuelven— lo resolvés vos con la IA.** Esa es la parte del ejercicio. Acá solo te marcamos *qué* problema atacar, no *cómo* resolverlo.

Qué tenés que lograr

Hacé **los dos helpers** marcados con  :

1. `respuestas-helper.js` — extraé el `res.status(...).send/json(...)` repetido en los controllers a funciones con nombre (`responderOk` , `responderNotFound` , `responderError` ...) en `src/helpers/` .
2. `fechas-helper.js` — sacá `calcularEdad` / `agregarEdad` de `alumnos-service.js` a `src/helpers/` y dejá que el service los **importe** en vez de definirlos adentro.
3. Reemplazá las repeticiones por llamadas al helper (controllers y service).
4. Que los endpoints respondan **exactamente igual** que antes: mismos status code, mismo body, y la **edad de cada alumno tiene que dar el mismo número**.

Cómo encarar el prompting

Prompt de diagnóstico primero:

Pegale `alumnos-controller.js` y `cursos-controller.js` y pedile: *"identificá qué lógica se repite y propondría extraer a un helper, sin escribir código todavía. Listame los candidatos ordenados por cuánto código ahorran."*

Después, ejecución:

Pedí **un helper a la vez**. Restricción clave: *"el helper tiene que ser una función pura / un módulo independiente, exportado con ES modules, y no debe cambiar la respuesta HTTP que ya devuelven los endpoints"*.

💡 **Tip:** pedile a la IA que te diga **dónde poner el helper** y **cómo nombrarlo** según la convención del proyecto (mirá cómo está hecho `LogHelper`: clase con métodos estáticos vs. funciones sueltas). Coherencia > "lo más moderno".

Verificación del resultado

- ☐ Los helpers están en `src/helpers/` y son `import`-ables (ES modules, no `require`).
- ☐ Cada endpoint que usa el helper quedó **más corto** y se lee mejor.
- ☐ Los **status codes no cambiaron**: probá happy path y casos de error (404, 400) en Postman.
- ☐ `calcularEdad` / `agregarEdad` ya **no están definidos dentro de** `alumnos-service.js`: viven en `src/helpers/fechas-helper.js` y el service los importa. La edad de los alumnos sigue dando el mismo número.
- ☐ El helper no quedó "atado" a una entidad puntual (es reutilizable por `cursos`, `materias`, etc.).

😬 Pregunta para el oral: *¿un helper de respuestas debería conocer los status codes, o se los deberías pasar el controller? ¿Por qué? ¿Dónde "vive" la decisión de devolver 404?*

Entrega

Bitácora + commit. En la reflexión, explicá qué **no** extrajiste a un helper y por qué (no todo lo que se repite conviene extraerlo — a veces dos cosas se parecen hoy pero van a divergir mañana).